SSD_Padmux Instructions for Use

1. What is Padmux

Padmux is a linux platform driver located in kernel\drivers\sstar\padmux. Because of the multiplexing of gpio pins, usually the same gpio can be used for different functions, so we use the padmux driver to uniformly configure the gpio pin functions that need to be used when booting.

2. PadMux driver introduction

The Padmux driver is defined as follows:

```
2. static struct platform_driver sstar_padmux_driver = {
3. .driver = {
4. .name = "padmux",
5. .owner = THIS_MODULE,
6. .of_match_table = sstar_padmux_of_match,
7. .pm = &sstar_padmux_pm_ops,
8. },
9. .probe = sstar_padmux_probe,
10. };
```

2.1. sstar_padmux_of_match

Used to match the gpio table device that needs to be configured at boot:

infinity2m-ssc011a-s01a-padmux-display.dtsi

```
17
18  #include <../../../../drivers/sstar/include/infinity2m/padmux.h>
19  #include <../../../../drivers/sstar/include/mdrv_puse.h>
20
21  / {
22      soc {
23          padmux {
24              compatible = "sstar-padmux";
25              schematic =
26                  //<PAD_GPIO0  >,
27                  <PAD_GPIO1          PINMUX_FOR_GPIO_MODE        MDRV_PUSE_I2C1_DEV_RESET >,
28                  <PAD_GPIO2          PINMUX_FOR_I2C1_MODE_1      MDRV_PUSE_I2C1_SCL >,
29                  <PAD_GPIO3          PINMUX_FOR_I2C1_MODE_1      MDRV_PUSE_I2C1_SDA >,
30                  <PAD_GPIO4          PINMUX_FOR_PWM0_MODE_3      MDRV_PUSE_PWM0 >,
31                  <PAD_GPIO5          PINMUX_FOR_PWM1_MODE_4      MDRV_PUSE_PWM1 >,
32                  <PAD_GPIO6          PINMUX_FOR_EJ_MODE_3        MDRV_PUSE_EJ_TDO >,
33                  <PAD_GPIO7          PINMUX_FOR_EJ_MODE_3        MDRV_PUSE_EJ_TDI >,
34                  //<PAD_GPIO8  >,
35                  //<PAD_GPIO9  >,
36                  //<PAD_GPIO10  >,
37                  <PAD_GPIO11         PINMUX_FOR_GPIO_MODE        MDRV_PUSE_I2CSW_SCL>,
38                  <PAD_GPIO12         PINMUX_FOR_GPIO_MODE        MDRV_PUSE_AIO_AMP_PWR>,
39                  <PAD_GPIO13         PINMUX_FOR_GPIO_MODE        MDRV_PUSE_I2C1_DEV_IRQ >,
40                  //<PAD_GPIO14  >,
41                  //<PAD_FUART_RX  >,
42                  //<PAD_FUART_TX       PINMUX_FOR_GPIO_MODE        MDRV_PUSE_UTMI_POWER>,
43                  <PAD_FUART_CTS      PINMUX_FOR_GPIO_MODE        MDRV_PUSE_CPUFREQ_VID0>,
44                  <PAD_FUART_RTS      PINMUX_FOR_GPIO_MODE        MDRV_PUSE_CPUFREQ_VID1>,
45                  <PAD_TTL0           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT00 >,
46                  <PAD_TTL1           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT01 >,
47                  <PAD_TTL2           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT02 >,
48                  <PAD_TTL3           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT03 >,
49                  <PAD_TTL4           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT04 >,
50                  <PAD_TTL5           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT05 >,
51                  <PAD_TTL6           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT06 >,
52                  <PAD_TTL7           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT07 >,
53                  <PAD_TTL8           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT08 >,
54                  <PAD_TTL9           PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT09 >,
55                  <PAD_TTL10          PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT10 >,
56                  <PAD_TTL11          PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT11 >,
57                  <PAD_TTL12          PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT12 >,
58                  <PAD_TTL13          PINMUX_FOR_TTL_MODE_1       MDRV_PUSE_TTL_DOUT13 >.
```

2.2. sstar_padmux_pm_ops

Implement the resume and suspend functions corresponding to str standby:

```
1. static const struct dev_pm_ops sstar_padmux_pm_ops = {
2. .suspend_late = sstar_padmux_suspend,
3. .resume_early = sstar_padmux_resume,
4. };
```

2.3. sstar_padmux_probe

After matching the padmux device, the system will call sstar_padmux_probe to configure the gpio function:

Finally, configure gpio to the corresponding function through MDrv_GPIO_PadVal_Set:

```
static int _mdrv_padmux_dts(struct device_node* np)
{
    int nPad;

    if (0 >= (nPad = of_property_count_elems_of_size(np, PADINFO_NAME, sizeof(pad_info_t))))
    {
        PAD_PRINT("[%s][%d] invalid dts of padmux.schematic\n", __FUNCTION__, __LINE__);
        return -1;
    }
    if (NULL == (_pPadInfo = kmalloc(nPad*sizeof(pad_info_t), GFP_KERNEL)))
    {
        PAD_PRINT("[%s][%d] kmalloc fail\n", __FUNCTION__, __LINE__);
        return -1;
    }
    if (of_property_read_u32_array(np, PADINFO_NAME, (u32*)_pPadInfo, nPad*sizeof(pad_info_t)/sizeof(U32)))
    {
        PAD_PRINT("[%s][%d] of_property_read_u32_array fail\n", __FUNCTION__, __LINE__);
        kfree(_pPadInfo);
        _pPadInfo = NULL;
        return -1;
    }
    _nPad = nPad;
#if 1
    {
        int i;
        PAD_PRINT("[%s][%d] ******************************\n", __FUNCTION__, __LINE__);
        for (i = 0; i < _nPad; i++)
        {
            PAD_PRINT("[%s][%d] (PadId, Mode, Puse) = (%d, 0x%02x, 0x%08x)\n", __FUNCTION__, __LINE__,
                _pPadInfo[i].u32PadId,
                _pPadInfo[i].u32Mode,
                _pPadInfo[i].u32Puse);
            MDrv_GPIO_PadVal_Set((U8)_pPadInfo[i].u32PadId & 0xFF, _pPadInfo[i].u32Mode);
        }
        PAD_PRINT("[%s][%d] ******************************\n", __FUNCTION__, __LINE__);
    }
#endif
    return 0;
} « end _mdrv_padmux_dts »
static int sstar_padmux_resume(struct device *dev)
{
    PAD_PRINT("%s\r\n", __FUNCTION__);
    _mdrv_padmux_dts(dev->of_node);
    return 0;
}
```

kernel\drivers\sstar\gpio mdrv_gpio.c

```
1.  int MDrv_GPIO_PadVal_Set(U8 u8IndexGPIO, U32 u32PadMode)
2.  {
3.      return MHal_GPIO_PadVal_Set(u8IndexGPIO, u32PadMode);
4.  }
```

kernel\drivers\sstar\gpio\infinity2m mhal_gpio.c

```
1.  int MHal_GPIO_PadVal_Set(U8 u8IndexGPIO, U32 u32PadMode)
2.  {
3.      return HalPadSetVal((U32)u8IndexGPIO, u32PadMode);
4.  }
```

kernel\drivers\sstar\gpio\infinity2m mhal_pinmux.c

```
1.  S32 HalPadSetVal(U32 u32PadID, U32 u32Mode)
2.  {
3.      if (FALSE == _HalCheckPin(u32PadID)) {
4.          return FALSE;
5.      }
6.      if(u32PadID>=PAD_ETH_RN && u32PadID <= PAD_SAR_GPIO3)
7.          return HalPadSetMode_MISC(u32PadID, u32Mode);
8.      return HalPadSetMode_General(u32PadID, u32Mode);
9.  }
10. static S32 HalPadSetMode_General(U32 u32PadID, U32 u32Mode)
11. {
12.     U32 u32RegAddr = 0;
13.     U16 u16RegVal  = 0;
14.     U8  u8ModeIsFind = 0;
15.     U16 i = 0;
16.
17.     for (i = 0; i < sizeof(m_stPadMuxTbl)/sizeof(struct
stPadMux); i++)
18.     {
19.         if (u32PadID == m_stPadMuxTbl[i].padID)
20.         {
21.             u32RegAddr = _RIUA_16BIT(m_stPadMuxTbl[i].base,
m_stPadMuxTbl[i].offset);
22.
23.             if (u32Mode == m_stPadMuxTbl[i].mode)
24.             {
25.                 u16RegVal = _GPIO_R_WORD_MASK(u32RegAddr,
0xFFFF);
26.                 u16RegVal &= ~(m_stPadMuxTbl[i].mask);
27.                 u16RegVal |= m_stPadMuxTbl[i].val; // CHECK
Multi-Pad Mode
28.
29.                 _GPIO_W_WORD_MASK(u32RegAddr, u16RegVal,
0xFFFF);
30.
31.                 u8ModeIsFind = 1;
32. #if (ENABLE_CHECK_ALL_PAD_CONFLICT == 0)
33.                 break;
34. #endif
35.             }
36.             else
37.             {   //Clear high priority setting
38.                 u16RegVal = _GPIO_R_WORD_MASK(u32RegAddr,
m_stPadMuxTbl[i].mask);
39.                 if (u16RegVal == m_stPadMuxTbl[i].val)
```

```
40.                 {
41.                     printk(KERN_INFO"[Padmux]reset PAD%d(reg
0x%x:%x; mask0x%x) t0 %s (org: %s)\n",
42.                             u32PadID,
43.                             m_stPadMuxTbl[i].base,
44.                             m_stPadMuxTbl[i].offset,
45.                             m_stPadMuxTbl[i].mask,
46.
m_stPadModeInfoTbl[u32Mode].u8PadName,
47.
m_stPadModeInfoTbl[m_stPadMuxTbl[i].mode].u8PadName);
48.                     if (m_stPadMuxTbl[i].val != 0)
49.                     {
50.                         _GPIO_W_WORD_MASK(u32RegAddr, 0,
m_stPadMuxTbl[i].mask);
51.                     }
52.                     else
53.                     {
54.                         _GPIO_W_WORD_MASK(u32RegAddr,
m_stPadMuxTbl[i].mask, m_stPadMuxTbl[i].mask);
55.                     }
56.                 }
57.             }
58.         }
59.     }
60.
61.     return (u8ModeIsFind) ? 0 : -1;
62. }
```

通过在m_stPadMuxTbl里面找到需要配置的gpio和mode设定对应的reg。

以PAD_GPIO4配置成PINMUX_FOR_PWM0_MODE_3为例：

当在dts padmux里面配置了：

```
<PAD_GPIO4      PINMUX_FOR_PWM0_MODE_3    MDRV_PUSE_PWM0 >,
```

在probe的时候会根据m_stPadMuxTbl的寄存器设定将寄存器bank 0x103c offset 0x65的bit2设置为1：

```
1.  #define CHIPTOP_BANK                            0x101E00
2.  #define REG_PWM0_MODE                           0x07
3.  #define REG_PWM0_MODE_MASK                      BIT0|BIT1|BIT2
4.  {PAD_GPIO4, CHIPTOP_BANK, REG_PWM0_MODE,  REG_PWM0_MODE_MASK,
BIT1|BIT0, PINMUX_FOR_PWM0_MODE_3},
```

可以在板子通过读取reg确认设定有没有生效：riu_r 0x101E 0xE 的bit0/bit1

---

3. PadMux怎么用

通过上面padmux驱动的介绍，我们知道只要确认padmux驱动有enable，并在dts里面将需要配置的pin和对应的mode写进去，开机padmux probe的时候就会将gpio对应的功能配置好。需要注意的是pamdux驱动依赖gpio驱动，所以这两个驱动都要build进kernel。目前公板默认的配置gpio和padmux都有默认打开buildin：

```
1.  CONFIG_MS_GPIO=y
2.  CONFIG_MS_PADMUX=y
```

下面介绍如何在dts的padmux中配置对应的gpio功能。

3.1. 用哪个padmux dts文件

根据下面的follow找到对应kernel config文件中指定的padmux dts文件，例如：

kernel\arch\arm\configs\infinity2m_spinand_ssc011a_s01a_minigui_defconfig:

```
1. CONFIG_SS_DTB_NAME="infinity2m-spinand-ssc011a-s01a-display"
```

kernel\arch\arm\boot\dts\infinity2m-spinand-ssc011a-s01a-display.dts:

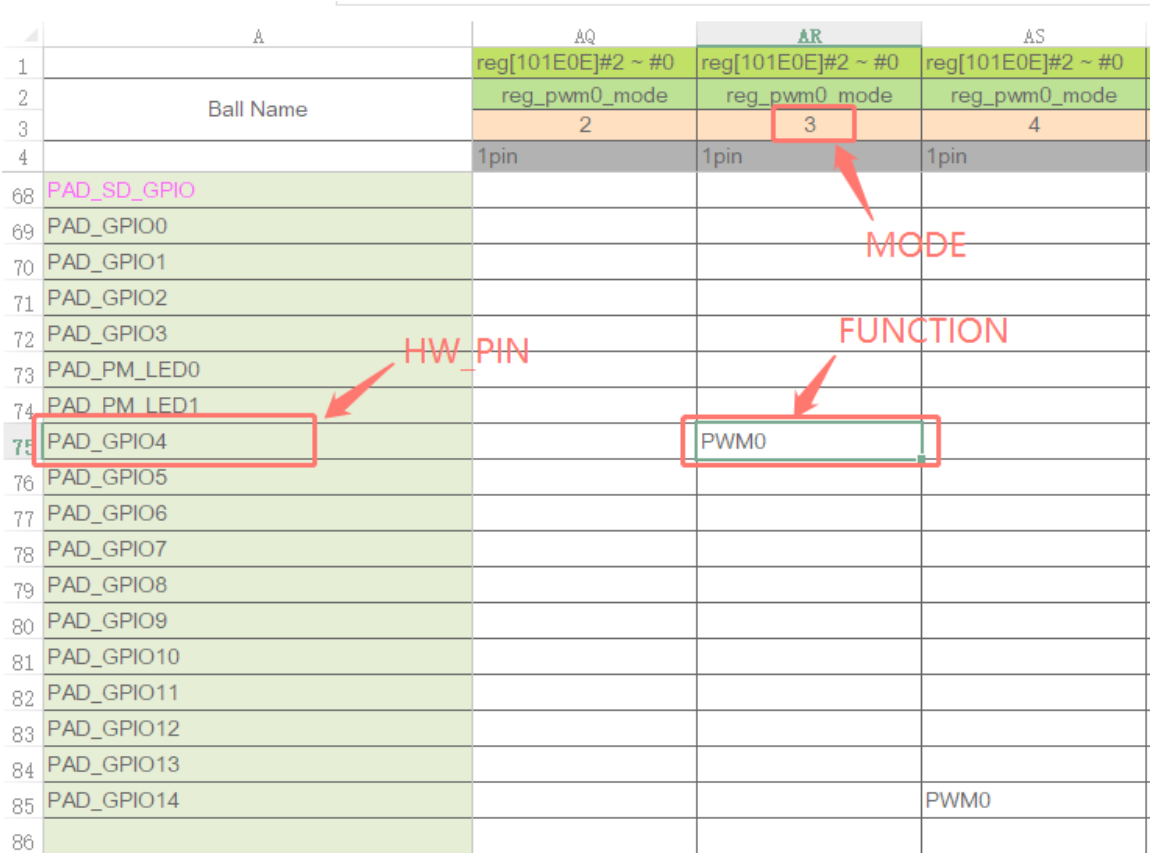
```
1. #include "infinity2m.dtsi"
2. #include "infinity2m-ssc011a-s01a-display.dtsi"
3. #include "infinity2m-ssc011a-s01a-padmux-display.dtsi"
```

kernel\arch\arm\boot\dts\infinity2m-ssc011a-s01a-padmux-display.dtsi

---

3.2. 配置gpio mode

根据SDK release的硬件发布资料SSD201 HW Checklist V18.xlsx，找到padmux这栏：



	A	AQ	AR	AS
1		reg[101E0E]#2 ~ #0	reg[101E0E]#2 ~ #0	reg[101E0E]#2 ~ #0
2	Ball Name	reg_pwm0_mode	reg_pwm0_mode	reg_pwm0_mode
3		2	3	4
4		1pin	1pin	1pin
68	PAD_SD_GPIO			
69	PAD_GPIO0			
70	PAD_GPIO1			
71	PAD_GPIO2			
72	PAD_GPIO3			
73	PAD_PM_LED0			
74	PAD_PM_LED1			
75	PAD_GPIO4		PWM0	
76	PAD_GPIO5			
77	PAD_GPIO6			
78	PAD_GPIO7			
79	PAD_GPIO8			
80	PAD_GPIO9			
81	PAD_GPIO10			
82	PAD_GPIO11			
83	PAD_GPIO12			
84	PAD_GPIO13			
85	PAD_GPIO14			PWM0
86				

以上图将PAD_GPIO4配置成PWM0为例，可以看到PAD_GPIO4要配置成PWM0只能将其配置成pwm0 mode 3，所以在padmux dts里面配置如下：

```
< PAD_GPIO4      PINMUX_FOR_PWM0_MODE_3    MDRV_PUSE_PWM0>,
```

其中MDRV_PUSE_PWM0一般只做说明注释没有特殊用途，除非对应功能驱动有特别设定。其他i2c,spi,uart,ttl,mipi的设定也可以参考hw checklist在padmux dts配置。

4. Precautions

The same gpio pin can only be multiplexed into one function, and cannot be configured into two functions in padmux at the same time. If the function configured in padmux does not take effect, you need to confirm whether it is configured with other functions in dts.